INTERNATIONAL TRANSACTIONS IN OPERATIONAL RESEARCH

INTERNATIONAL TRANSACTIONS IN OPERATIONAL RESEARCH

Intl. Trans. in Op. Res. 20 (2013) 627–643
DOI: 10.1111/itor.12025

A heuristic and an exact method for the gate matrix connection cost minimization problem

L. De Giovanni[a], G. Massi[b], F. Pezzella[b], M.E. Pfetsch[c], G. Rinaldi[d] and P. Ventura[d]

[a] *Dipartimento di Matematica, Università degli studi di Padova, via Trieste, 63, 35121Padova, Italy*
[b] *Dipartimento di Ingegneria dell'Informazione, Università Politecnica delle Marche – via Brecce Bianche 12, Ancona, Italy*
[c] *Research Group Optimization, Department of Mathematics, Technische Universität Darmstadt, Dolivosstraße 15, 64293 Darmstadt, Germany*
[d] *Istituto di Analisi dei Sistemi ed Informatica "Antonio Ruberti" del CNR, viale Manzoni 30, 00185 Roma, Italy*
E-mail: luigi@math.unipd.it [De Giovanni]; massi@dii.univpm.it [Massi]; pezzella@dii.univpm.it [Pezzella]; pfetsch@mathematik.tu-darmstadt.de [Pfetsch]; rinaldi@iasi.cnr.it [Rinaldi]; ventura@iasi.cnr.it [Ventura]

Received 15 October 2011; received in revised form 19 April 2013; accepted 22 April 2013

Abstract

In many applications, a sequencing of patterns (electronic circuit nodes, cutting patterns, product orders, etc.) has to be found in order to optimize some given objective function, giving rise to the so-called open stack problems. We focus on a problem related to the optimization of gate matrix layouts: electronic circuits are obtained by connecting gates and one seeks a gate layout permutation that minimizes connection costs under restrictions on the circuit area. In the literature, the connection costs and circuit area are also known as time of open stacks and maximum number of open stacks, respectively. We propose a genetic algorithm providing heuristic solutions and a branch-and-cut algorithm based on a new linear integer programming formulation that represents, to the best of our knowledge, the first exact method proposed in the literature. The algorithms have been tested on real instances and on data sets from the literature. The computational results give evidence that the proposed methods provide solutions that improve the ones found by the approaches presented in the literature.

Keywords: time of open stacks; maximum number of open stacks; pattern sequencing; genetic algorithms; dynamic fitness function; integer linear programming; branch-and-cut

1. Introduction

The gate matrix layout problem is related to programmable logic array folding in very large scale integration (VLSI) electronic circuit design (Möhring, 1990). Roughly speaking, "gates" correspond to circuit nodes, and different connections between them are required. Each connection involves a

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies
Published by John Wiley & Sons Ltd, 9600 Garsington Road, Oxford, OX4 2DQ, UK and 350 Main St, Malden, MA02148, USA.

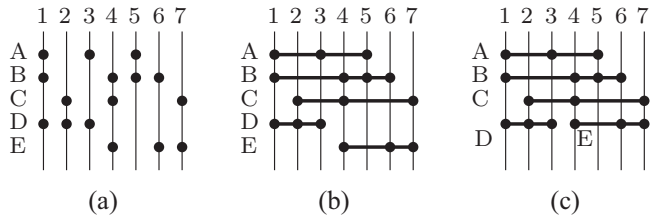


Fig. 1. Sample gate matrix: connection requests (a), wired nets (b), and connection tracks (c).

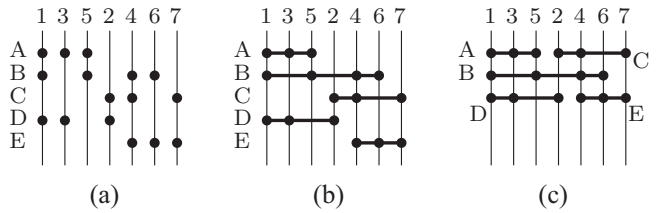


Fig. 2. Sample gate matrix: an improved gate sequence.

subset of nodes and is called "net." Fig. 1(a) shows an example where seven gates (vertical lines) have to be connected according to five different nets, described by dots of the same row: net *A* connects gates 1, 3, and 5; net *B* connects gates 1, 4, 5, and 6, etc. "Wires" are used to create connections, one for each net, as shown by horizontal lines in Fig. 1(b).

Note that to connect the gates of a net, it may be necessary to cross other gates not included in the net, depending on the gate layout sequence. Also, a single "connection track" can be used to place nonoverlapping net wires, as shown in Fig. 1(c) for nets *D* and *E*. The total wire length determines the connection cost, whereas the number of tracks determines the total circuit area, which may be limited by design constraints or efficiency issues. Both the indicators give an estimate of the circuit layout efficiency and depend on how gates are sequenced. The gate layout of Fig. 1 requires 19 wire units and 4 tracks, corresponding to the maximum number of overlapping net wires. A better layout is shown in Fig. 2, using 15 wire units and 3 tracks.

We define the Gate Matrix Connection Cost Minimization Problem (GMCCP) as the problem of finding a gate permutation such that the connection cost is minimized and the number of required tracks is limited. The problem is related to the gate matrix layout problem (Möhring, 1990), where the number of required tracks is minimized. GMCCP is NP-hard and has several applications in different fields (Linhares and Yanasse, 2002). For example, in production planning, gates correspond to articles, nets to client orders (subsets of articles), and wires represent the occupation of dedicated order stacks (and related loading facilities) over all the order processing time, depending on the article production sequence. The same stack can be used for nonoverlapping orders, and one wants to find a production sequence that minimizes the total stack occupation time, under the restriction that the maximum number of overlapping orders, that is the maximum number of simultaneously open stacks during the production process, is at most the number of available stacks, as determined by the plant layout. Similarly, in cutting stock contexts, the items (corresponding to nets in GMCCP) obtained from panels sawed according to given cutting patterns (corresponding to gates) are heaped on stacks around the sawing machine. Stacks remain open during all the production time of the

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

related item and, again, the same stack (corresponding to a track) can be used for items whose production does not overlap over time. The problem is to find a cutting pattern permutation that minimizes the total stack opening time, provided that the maximum number of simultaneously open stacks during the cutting process does not exceed a given threshold, which is a parameter of the sawing center. In the literature, the total stack occupation time and the maximum number of simultaneously open stacks are known as Time of Open Stacks (TOS) and Maximum number of Open Stacks (MOS), respectively. In GMCCP, the wire length corresponds to TOS, and the number of required tracks corresponds to MOS. Note that a given gate sequence may not be feasible because the number of "required" tracks (MOS) exceeds the number of "available" tracks as determined by the restrictions on the circuit area.

We can characterize an instance of GMCCP by a "production matrix" $M \in \{0, 1\}^{m \times n}$ and a parameter $\lambda \in \mathbb{Z}_+$ representing the number of "available" tracks and, hence, an upper bound for MOS, meaning that all sequences having MOS greater than λ are not feasible. Rows of M are associated with nets, columns with gates, and $M(i, j) = 1$ if and only if net i includes gate j. A solution of GMCCP consists in a sequence $\phi : [1, \ldots, n] \to [1, \ldots, n]$, where $\phi(j)$ indicates the layout position of gate j. Such a solution defines a new matrix M_ϕ obtained from M by permuting its columns according to ϕ. From M_ϕ, we obtain a "stack matrix" $\bar{M}_\phi$ by switching to 1 any 0 of M_ϕ between two 1's in the same row.

Therefore $\bar{M}_\phi(i, j) = 1$ if and only if, according to ϕ, the wire of net i includes or crosses gate j. Figure 3 shows the production matrix of the sample gate matrix of Fig. 1(a) and the stack matrices of the sequences [1, 2, 3, 4, 5, 6, 7] and [1, 3, 5, 2, 4, 6, 7]. Note that the values MOS and TOS for a given sequence ϕ can easily be obtained from $\bar{M}_\phi$. The length of the wire required by net i is the distance (in number of gates) between the first and the last gates of i, equal to the number of 1's in the ith row of $\bar{M}_\phi$, minus 1 (the first gate must not be considered). Therefore, the length of the wire for a single net is the sum of the entries of the related row of $\bar{M}_\phi$ minus 1, and TOS is the sum of all the entries of $\bar{M}_\phi$ minus m. MOS is the maximum number of 1's appearing in any of the columns of $\bar{M}_\phi$. Summarizing, given a matrix $M \in \{0, 1\}^{m \times n}$ and a threshold $\lambda \in \mathbb{Z}_+$, GMCCP is to find a column permutation having MOS not greater than λ and minimum TOS.

Literature on pattern-sequencing problems is rich and related to different application fields and solution techniques. Nevertheless, most authors consider only MOS minimization (Becceneri et al., 2004; Chu and Stuckey, 2009; Hu and Chen, 1990; Lopes and Valério de Carvalho, 2010; Smith and Gent, 2005, among others), and TOS is sometimes used to heuristically drive the search of good MOS-minimizing sequences (e.g., De Giovanni et al., 2013; Oliveira and Lorena, 2006). Note that minimizing MOS is not equivalent to minimizing TOS (Linhares and Yanasse, 2002). Among the papers that explicitly take TOS optimization into account, the earliest ones are mainly concerned with heuristics (e.g., Dyson and Gregory, 1974; Madsen, 1988), whereas the most recent ones propose metaheuristics: for example, Förster and Wäscher (1998) present a simulated annealing based on a TSP-like (where TSP is traveling salesman problem) 3-OPT neighborhood; Oliveira and Lorena (2002) propose a constructive genetic algorithm (CGA) integrating genetic operators, local search, and schemata filling heuristics; Respício and Captivo (2005) consider a biobjective approach for an application in paper industry where the set of Pareto-optimal solutions is approximated by a genetic algorithm improved by initial heuristics and local search.

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

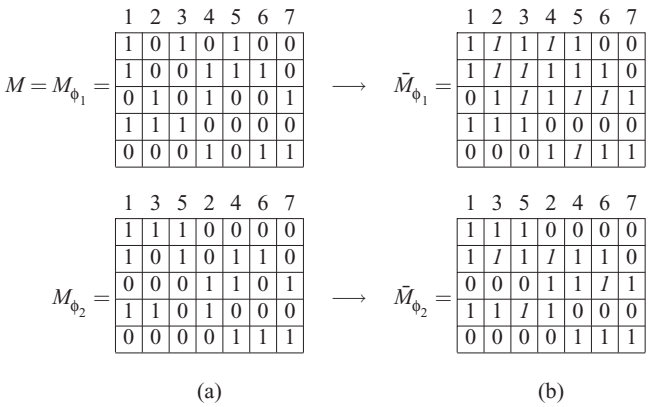


Fig. 3. Sample production matrix (a) and stack matrices for the sequences $\phi_1 = [1, 2, 3, 4, 5, 6, 7]$ and $\phi_2 = [1, 3, 5, 2, 4, 6, 7]$, with switched elements in italics (b).

In this paper, we focus on GMCCP as defined previously, that is, on pattern-sequencing problems where TOS has to be minimized and further restrictions on MOS have to be satisfied. We propose two algorithms: the first one aims at determining both the least possible threshold λ for the number of tracks (MOS), and a feasible sequence with a low connection cost (TOS); the second one starts from this sequence and minimizes the wire length (TOS), provided that the MOS value does not exceed λ. The first algorithm is a heuristic based on a genetic approach with a composite and dynamic definition of the fitness function. The second algorithm exploits the flexibility of a new integer programming formulation based on the properties of consecutive-ones matrices. Such a formulation is then solved by a branch-and-cut (B&C) algorithm. The proposed methods have been tested on instances from VLSI and woodcutting industries as well as on data sets available in the literature.

2. Genetic algorithm

The aim of the first algorithm for GMCCP is twofold. First, we need to determine a threshold λ for MOS, which may not be known *a priori*. For example, in production or cutting stock environments, the limitation on the number of available stacks may be too restrictive, so that no feasible sequence exists and temporary warehousing is necessary (it might be the case that the stacks in excess have to be moved to a temporary warehouse far away from the loading facilities or from the cutting machinery). We thus want to take λ as low as possible to limit temporary warehousing and preserve process efficiency. Second, we seek a feasible sequence that, beyond having a MOS value that obeys the threshold λ, also provides a good TOS value. Note that this may also be useful in the B&C algorithm for TOS optimization, since it provides a good initial incumbent solution.

We consider a genetic approach: genetic algorithms iteratively evolve a population of several individuals according to the principle of natural selection. Each individual encodes a particular solution and, at each generation, new individuals are obtained by selecting parents and combining their features. Aiming at obtaining better and better solutions, a fitness value is associated to each

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

1. Determine individuals of the **initial population**
2. *Repeat* (for each generation)
3. *Repeat* (for each offspring)
4. **Select** two parents
5. Generate offspring by **crossover**
6. Apply **mutation** to offspring
7. *Until* a set of new individuals are generated
8. **Replace** old individuals with new ones
9. **Refine** the fittest individuals by local search
10. **Iteratively adapt** fitness criteria to intensification/diversification
11. *Until* termination conditions are satisfied
12. *Return* the best individual found.

Fig. 4. Sketch of the genetic algorithm for GMCCP.

individual: the fitter the individuals the more likely they are to be selected as parents and to transmit their features to new generations. The genetic algorithm for GMCCP (GAG) is sketched in Fig. 4. Individuals are encoded as sequences of columns, and the initial population is obtained in part heuristically (Yuen, 1995) and in part by random column permutations (Step 1). The operator that recombines individuals and obtains offspring for the new generation (Steps 3–7) is the two-points order crossover, borrowed from the literature on the TSP (Oliver et al., 1987). After selecting two parents, two new individuals are generated: two sequence cut points are randomly chosen and each individual inherits the external subsequences from one parent, whereas the central elements are filled-in in the order relative to the other parent. To avoid premature convergence, the selection operator chooses parents at random, giving priority to the fittest individuals: one parent among the best π_B (where π_B is a parameter) and the other one among all the individuals. The parameter π_B, as well as all the other parameters that we will describe in this paper, has been chosen according to computational experiments to trade-off efficiency and quality of the results (see Section 4). Also, new individuals undergo a mutation with a given probability π_M: mutation exchanges the position of two randomly chosen columns. The new generation is obtained in Step 8 by putting the new offspring in place of all but a set of π_E"elite" individuals. The elite individuals are chosen at random, giving priority to the fittest ones: one half among the best π_B and one half among all the individuals. Before starting the next generation, a refinement operator explores the 2-OPT neighborhood (Lin and Kernighan, 1973) of most promising individuals and replaces them by local optima evaluated by the fitness function (Step 9). GAG stops after a fixed number of generations, returning the best individual found so far.

While preserving the general schema of standard genetic algorithms, GAG introduces some new features, which have experimentally shown to significantly impact on its performance. They are mainly related to the fitness function definition, implementation of an iterative fitness-driven intensification/diversification search scheme, refinement operator, and incremental evaluation of the fitness for the solution generated by crossover and local search operators. The fitness function is used to guide the selection mechanism and, according to the twofold aim of GAG, both MOS and TOS have to be taken into account, together with the peculiarities of the solution space landscape. In fact, MOS is related to critical subsequences and is very unlikely to change under small sequence

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

perturbations. Further indicators are thus necessary to discriminate fittest individuals and, as discussed in De Giovanni et al. (2013), TOS is not enough: both MOS and TOS measure the whole sequence and may hide good "local" features. We thus propose the new indicators *NEW* and *IOS*. They are based on relations between close-by columns in a given sequence ϕ and are defined as follows:

$$NEW(\phi) = \sum_{j=2}^{n} \sum_{i=1}^{m} M_\phi(i, j) \cdot (M_\phi(i, j) - M_\phi(i, j-1)), \tag{1}$$

$$IOS(\phi) = \max_{j=2\ldots n} \left\{ 0, \sum_{i=1}^{m} \bar{M}_\phi(i, j) - \sum_{i=1}^{m} \bar{M}_\phi(i, j-1) \right\}. \tag{2}$$

Indicator *NEW* sums up the 1's in one column of M_ϕ not contained in the previous one and can be seen as a local counterpart of TOS: small values point out sequences with similar consecutive gate patterns, which means that (in a local perspective) a few 1's have to be added to obtain the stack matrix (TOS tends to be small). Indicator *IOS* is the maximum increment in the number of 1's from one column of $\bar{M}_\phi$ to the following one and it is the local counterpart of MOS: it favors a regular utilization of stacks, penalizing sequences with local peaks in the number of open stacks. Summarizing, the fitness of an individual is a weighted sum of MOS, TOS, *NEW*, and *IOS*, and it is supposed to both point out solutions with good MOS and/or TOS, and direct the search toward promising regions of the solution space.

We have also implemented intensification and diversification search phases, as it is usual, in general, in genetic algorithms and metaheuristics. Instead of implementing further algorithmic components as in previous literature approaches, we propose to obtain intensification and diversification search phases by dynamically changing the weights during the evolution (Step 10). The values adopted for the weights and the way they are changed are detailed in Section 4 (see Table 1) as the result of the parameter calibration. In general, the following schema is considered. Three weight settings are defined: Setting (I), giving emphasis to MOS and, in second order, to TOS, with relatively small contribution of *IOS* and TOS; Setting (II), focusing on TOS and local features (*IOS* and *NEW*), with a small weight on MOS; and Setting (III), with a large MOS weight, a small contribution of TOS and no contribution of *IOS* or TOS. Setting (I) is related to the first generations and aims at finding individuals with good MOS and TOS values, using also the local indicators to deal with MOS permutative redundancy and preserve good local features. Setting (II) has the role of obtaining individuals with good TOS. Emphasis is also on *IOS* and *NEW*, and this allows to diversify the population, as individuals with different good local features are selected. Finally, Setting (III) tries to minimize the MOS value and can be seen as a search intensification. In the general schema, after the first generations with Setting (I), GAG iterates between Settings (II) and (III) to alternate the diversification and the intensification phases. The intensification phase is always executed last, to minimize the threshold λ (one of the aims of GAG) while finding a good feasible solution.

For the refinement operator, a standard implementation of the 2-OPT local search can be computationally expensive. Several speeding-up tricks have been devised: (i) the refinement operator is not applied at every generation, but once every π_R generations (the frequency is taken to be as low as

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

possible); (ii) just one individual of the population is refined (the fittest one); (iii) at most π_D descent moves per local search are allowed, so that it can stop before a local optimum is reached and just return an improved solution; (iv) while the standard 2-OPT strategy defines a neighbor solution by reversing any possible column subsequence, only subsequences of at most π_S columns are reversed. For example, starting from a sequence [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] and considering $\pi_S = 3$, the 2-OPT neighbor [1, 2, 6, 5, 4, 3, 7, 8, 9, 10] is not considered, as the reversed subsequence contains four columns. The refinement-related parameters have been set by calibration, as detailed in Section 4. Furthermore, an incremental neighbor evaluation has been implemented, based on the following invariance property of the stack matrix and the observation that the fitness function related to a sequence ϕ depends on TOS, MOS, *IOS*, which can be computed from the stack matrix $\bar{M}_\phi$, and on *NEW*, derived from the matrix M_ϕ.

Property: *Consider two column sequences ϕ and ψ, with $\phi = [\phi^A, \phi^B, \phi^C]$ (ϕ^A, ϕ^B, and ϕ^C are column subsequences), and $\psi = [\phi^A, \psi^B, \phi^C]$, where ψ^B is any permutation of ϕ^B. Then, (i) the first $|\phi^A|$ columns of $\bar{M}_\phi$ are equal to the ones of $\bar{M}_\psi$, and (ii) the last $|\phi^C|$ columns of $\bar{M}_\phi$ are equal to the ones of $\bar{M}_\psi$.*

Proof. A possible formula to build the stack matrix $\bar{M}_\psi$ is the following:

$$\bar{M}_\psi(i,j) = \begin{cases} 1 & \text{if } [M_\psi(i,j) = 1] \text{ or } [1 \le \sum_{p=1}^{j} M_\psi(i,p) < \sum_{p=1}^{n} M(i,p)] \\ 0 & \text{otherwise.} \end{cases}$$

The formula depends on the term $\sum_{p=1}^{n} M(i,p)$ (the number of 1's in the row i of the production matrix M), and on the number of 1's in row i in matrix M_ψ up to position j, which does not depend on the order of the elements of ψ up to the position j. For any position $j > |\phi^A| + |\psi^B| = |\phi^A| + |\phi^B|$ (i.e., the positions related to ϕ^C), the elements up to j are the same for both ϕ and ψ (just in a different position) and we have $\bar{M}_\phi(i,j) = \bar{M}_\psi(i,j)$, which proves (ii). To prove (i), we apply similar arguments after observing that we can build the same stack matrix starting from the last column up to the first one and, hence,

$$\bar{M}_\psi(i,j) = \begin{cases} 1 & \text{if } [M_\psi(i,j) = 1] \text{ or } [1 \le \sum_{p=j}^{n} M_\psi(i,p) < \sum_{p=1}^{n} M(i,p)], \\ 0 & \text{otherwise.} \end{cases}$$

□

The production matrix and, thanks to Property 1, the stack matrix used to compute the fitness of a neighbor solution can be obtained from the matrices of the center solution by modifying just the columns of the reversed subsequence. Note that the same incremental evaluation applies to offspring evaluation.

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

3. Exact branch-and-cut procedure

To explain our exact solution approach, we first describe our integer linear programming formulation, which is based on the notion of "consecutive-ones properties" for {0, 1}-matrices. A {0, 1}-matrix A has the "consecutive-ones property for rows" (or, briefly, A is *C1P*) if the columns of A can be permuted so as to obtain a "strict" C1P matrix, that is a {0, 1}-matrix such that in each row the ones appear consecutively (in each row no 0 separates two 1's). According to this definition, we can now state our formulation for GMCCP as follows: given $M \in \{0,1\}^{m\times n}$ and $\lambda \in \mathbb{Z}_+$, minimize $\sum_{i=1}^{m}\sum_{j=1}^{n} X(i,j)$ with

$$X \text{ is C1P} \tag{3}$$

$$X(i,j) \geq M(i,j),\ \forall i \in 1,\ldots,m,\ \forall j \in 1,\ldots,n \tag{4}$$

$$\sum_{i=1}^{m} X(i,j) \leq \lambda,\ \forall j \in 1,\ldots,n \tag{5}$$

$$X \in \{0,1\}^{m\times n}. \tag{6}$$

A feasible solution X of the previous system is then a {0, 1}-matrix (Constraint 6), obtained by turning 0's of M into 1's (Constraints 4), and such that there exists a sequence ϕ of its columns that defines a strict C1P matrix (Constraint 3).

Constraints 5 ensure that the MOS related to the solution X does not exceed the given value λ and the objective function corresponds to TOS. Still, in order to obtain an integer linear program, we have to translate Constraint 3 into linear inequalities.

Tucker (1972) gave a characterization of the C1P matrices in terms of excluded minors. A "minor" of a matrix $A \in \mathbb{R}^{m\times n}$ is the submatrix A_{IJ} of A defined by the ordered subsets I and J of its rows and columns, respectively. Characterizing all C1P matrices in terms of excluded minors means to give a minimal collection of minimal minors that are forbidden in any C1P matrix, where "minimal" for a collection is meant in the sense of set inclusion while "minimal" for a minor means that it does not have any other member of the collection as a minor. The collection provided in Tucker (1972) is made of the five special matrices T_k^1, T_k^2, T_k^3, T^4, T^5, called the "Tucker matrices." In particular, T^4 and T^5 have a fixed dimension, while T_k^1, T_k^2, and T_k^3 have a dimension depending on parameter k (see Fig. 5). Tucker proved that a matrix $A \in \{0,1\}^{m\times n}$ is C1P if and only if none of its minors is a Tucker matrix.

More recently, Oswald and Reinelt used the Tucker characterization in order to provide a description of the C1P matrices in terms of linear integer programming. Indeed, they first defined the {0, 1, −1}-matrices F_k^1, F_k^2, F^3, and F^4 (see Fig. 6) and proved the following theorem, where the inner product of two matrices $A, B \in \mathbb{R}^{m\times n}$, that is the scalar value $\sum_{i=1}^{m}\sum_{j=1}^{n} A(i,j)B(i,j)$, is denoted by $\langle A, B\rangle$ and where the set of all minors A_{IJ} of a matrix A, with $|I| = p$ and $|J| = q$, is denoted by $[A]^{p,q}$.

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

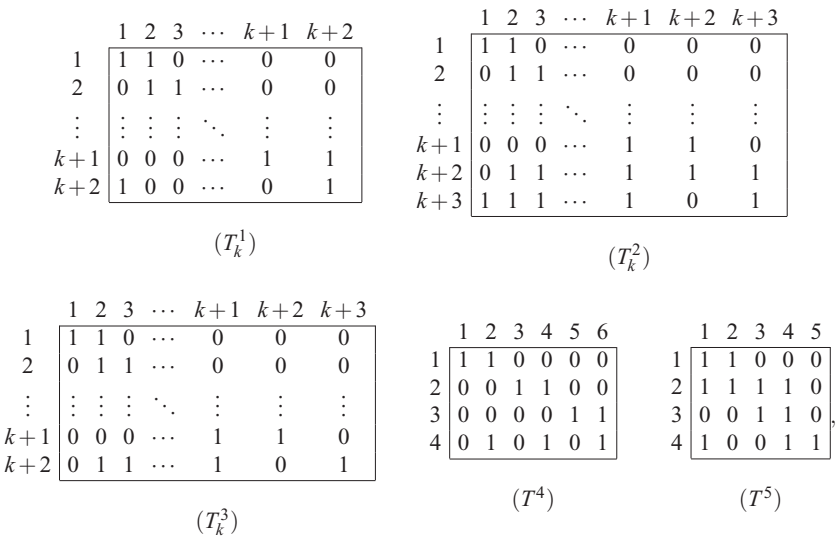


Fig. 5. Tucker matrices. Here k is any integer greater or equal to 1.

Theorem 1. (Oswald and Reinelt (2003, 2004)). *A matrix $X \in \{0,1\}^{m\times n}$ is C1P if and only if all the following Oswald and Reinelt (OR) inequalities are satisfied:*

$$\langle F_k^1, X_{IJ}\rangle \leq 2k+3,\ \forall\ X_{IJ} \in [X]^{k+2,k+2},\ \forall\ k \geq 1 \tag{7}$$

$$\langle F_k^2, X_{IJ}\rangle \leq 2k+3,\ \forall\ X_{IJ} \in [X]^{k+2,k+3},\ \forall\ k \geq 1 \tag{8}$$

$$\langle F^3, X_{IJ}\rangle \leq 8,\ \forall\ X_{IJ} \in [X]^{4,6} \tag{9}$$

$$\langle F^4, X_{IJ}\rangle \leq 8,\ \forall\ X_{IJ} \in [X]^{4,5} \tag{10}$$

where the left-hand side of the inequalities is defined by the inner product of minor X_{IJ} of X with a matrix of type F_k^1, F_k^2, F^3, or F^4 of the corresponding dimensions.

We can then use such a characterization to get a linear integer formulation of GMCCP by replacing constraint 3 with the set of inequalities 7–10. Observe that here, differently from the formulation proposed by Baptiste (2005), one does not need to take the order of the columns of X explicitly into account. Let X^* be the optimal solution of such a linear integer optimization program. Then X^* is a C1P matrix and one can apply the so-called PQ-tree algorithm (Booth and Lueker, 1976) that, in linear time, returns a columns sequence ϕ^* that turns X^* into a strict C1P matrix.

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

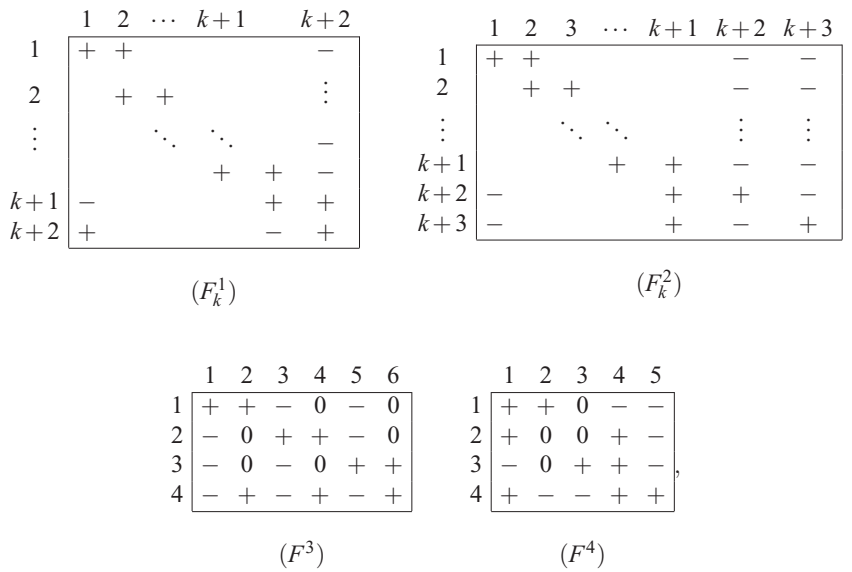


Fig. 6. Oswald–Reinelt matrices F_k^1 of size $(k+2)\times(k+2)$, F_k^2 of size $(k+2)\times(k+3)$, F^3 of size 4×6, and F^4 of size 4×5. Here "+" stands for "+1", "−" for "−1" and empty cells represent 0's.

Observe that, corresponding to the number of minors of the input matrix M, the number of constraints 7 and 8 is

$$\sum_{k=1}^{\min\{m-2,n-2\}} \left[(k+2)!\binom{m}{k+2}(k+2)!\binom{n}{k+2}\right]$$

and

$$\sum_{k=1}^{\min\{m-2,n-3\}} \left[(k+2)!\binom{m}{k+2}(k+3)!\binom{n}{k+3}\right],$$

respectively, and it grows exponentially with the size of M. Instead, the number of inequalities of types 9 and 10, that is, $4!\binom{m}{4}6!\binom{n}{6}$ and $4!\binom{m}{4}5!\binom{n}{5}$, respectively, even if not exponential, is bounded by a high polynomial in m and n. This implies that the linear relaxation of the proposed formulation can only be tackled in practice via a cutting plane procedure and, to this end, Oswald and Reinelt (2004) also defined a polynomial time algorithm to exactly separate inequalities (7)–(10).

In order to solve the linear integer problem we defined to optimality, here we implemented a B&C algorithm, based on a heuristic separation of the OR inequalities to solve the linear relaxation. The separation procedure is similar to the one proposed by Oswald and Reinelt (2003). In particular, such routine takes advantage from the fact that separating inequalities 7–10 over an integer point is much easier than separating them over a fractional point, since in the former case one can directly use the PQ-tree algorithm. Therefore, given a fractional solution $\widetilde{X}$, we round its values to the corresponding closest integers to obtain the matrix $\widehat{X}$ and then, using the PQ-tree algorithm, we check if $\widehat{X}$ is C1P.

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

In case it is not, we use the PQ-tree algorithm and iteratively remove rows and columns of $\widehat{X}$ until we find a Tucker matrix as a minor of $\widehat{X}$ and a corresponding OR inequality violated by $\widehat{X}$: if such inequality is also violated by $\widetilde{X}$, we use it as a cutting plane, otherwise we discard it. Observe that $\widehat{X}$ may not be feasible for the original problem but, since it is used only in order to easily separate the current fractional point, this does not affect the value of the optimal solution provided by the B&C algorithm. Also, if $\widehat{X}$ is C1P but it is not feasible for the original problem, then the PQ-tree algorithm will not be able to identify a violated inequality and the algorithm will proceed with branching.

Indeed, despite the fact that the separation routine we implemented is not exact (OR inequalities could exist violated by the current fractional point $\widetilde{X}$ that we are not able to find because they are satisfied by $\widehat{X}$), all the integer solutions that do not correspond to C1P matrices are cut off (observe that, if $\widetilde{X} \in \mathbb{Z}^{m \times n}$, then $\widehat{X} = \widetilde{X}$). This implies that the solution eventually provided by the B&C algorithm described previously is the optimal solution of the GMCCP instance given as input.

Finally, for the branching phase, instead of the standard 0-1 branching on the value of a variable $X(i, j)$, we implemented an alternative enumeration procedure based on betweenness considerations among the columns of the input matrix M. Therefore, in order to branch the current node of the enumeration tree, we perform the following: we fix a triple t, u, v of columns of M and then produce three subtrees T^t_{uv}, T^u_{tv}, and T^v_{tu}, such that T^t_{uv} (T^u_{tv}, T^v_{tu}, respectively) contains all the feasible C1P matrices X having column t (u, v, respectively) lying between columns u and v (t and v, t and u, respectively) in any of the columns sequences that turn X into a strict C1P matrix. Hence, we can produce T^t_{uv} (T^u_{tv}, T^v_{tu}, respectively) by taking into account that all column sequences in the subtree respect the constraints $X(i, t) = 1$ ($X(i, u) = 1$, $X(i, v) = 1$, respectively) whenever $M(i, u) = M(i, v) = 1$ ($M(i, t) = M(i, v) = 1$, $M(i, t) = M(i, u) = 1$, respectively), for all $1 \leq i \leq m$.

4. Computational results

The proposed approach for GMCCP has been implemented in C++ and, for the B&C procedure, we used the SCIP 2.1.1 framework (Achterberg, 2009), and Cplex 12.4 as linear programming solver. GAG has been run on a 3.2 GHz Intel i7-960 processor, and the B&C procedure on a 1.6 GHz Intel Centrino processor.

The algorithms have been tested on 330 instances from different data sets. The first one is a set of 11 instances from the VLSI industry, proposed by Hu and Chen (1990). Further 24 instances have been provided from two companies (denoted by "A" and "B") of the woodcutting industry (SCOOP Team, 2009). These real-world instances have different sizes: the number of rows and columns of the related production matrices ranges from 10 to 202 and 10 to 141, respectively. The remaining instances are taken from the literature on the "minimization of open stacks problem": 250 instances from Faggioli and Bentivoglio (1998), with $9 \leq m \leq 50$ and $10 \leq n \leq 30$, and two data sets from Smith and Gent (2005), namely the 25 Shaw instances ($m = n = 20$) and the Wilson instances ($10 \leq m \leq 100$ and $20 \leq n \leq 100$).

For GAG parameters, we experimentally set their values to the best trade-off we have found, during a preliminary calibration, between running times and quality of the provided solutions. In

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

Table 1
GAG fitness function weight settings

Setting	Total generations (%)	MOS	TOS	NEW	IOS
(I)	33	0.80	0.10	0.05	0.05
(II)	10 } × 2	0.10	0.50	0.20	0.20
(III)	23 }	0.95	0.05	0.00	0.00

Table 2
Results on VLSI instances

		CGA					GAG					B&C		
Instance ($m \times n$)	λ	SR_λ (%)	TOS	Avg	Dev (%)	T(s)[a]	SR_λ	TOS	Avg	Dev (%)	T(s)[b]	TOS	T(s)	Cuts
Wli (11×10)	4	100	*18*	18.0	0.0	0.5	100	**24**	24.0	0.0	0.0	**24**	5	171
Wsn (17×25)	8	100	104	106.6	3.6	1.5	100	96	96.4	0.6	0.5	–	–	1,426,560
V4000 (10×17)	5	100	*53*	53.3	1.7	0.5	60	**56**	57.2	2.9	0.2	**56**	42	23,716
V4050 (13×16)	5	100	41	41.4	1.3	0.5	100	**38**	38.0	0.0	0.2	**38**	23	793
V4090 (23×27)	10	90	95	96.8	1.7	2.0	100	109	109.0	0.0	0.8	–	–	318,964
V4470 (37×47)	9	100	246	262.4	5.6	66.5	100	237	237.2	0.2	4.2	–	–	521,980
X0 (40×48)	11	80	303	305.2	0.6	75.6	100	298	301.0	0.6	5.3	–	–	234,072
W1 (18×21)	4	100	**39**	39.8	4.6	1.0	100	**39**	39.0	0.0	0.3	**39**	4	694
W2 (48×33)	14	100	235	257.2	8.5	18.5	100	233	234.4	0.4	2.0	–	–	273,896
W3 (84×70)	18	50	677	751.6	11.9	306.3	100	668	673.0	0.5	21.6	–	–	304,388
W4 (202×141)	27	30	1730	1805.0	3.3	5224.7	60	1683	1879.4	7.5	29.1	–	–	98,724

–, no optimal solution or improvement after 1-hour computation. [a]On a 266 MHz Intel Pentium processor. [b]On a 3.2 GHz Intel i7-960 processor.

particular, we have fixed the number of generations to min{30n, 750} and the number of individuals to min{10n, 500}. Note that, in order to keep the running time as small as possible, in particular for smaller instances, these parameters depend on the number of columns n, with a maximum value. The fitness function weight settings are shown in Table 1: GAG uses Setting (I) for the first 33% generations and then iterates twice between Settings (II) and (III), each time for a number of generations equal to 10% and 23%, respectively. Computational evidence shows that, as far as the solution quality is concerned, the refinement operator should be applied as much as possible and, on the other hand, the required computational effort becomes considerable and grows up with instance size (in particular with n). Hence, the refinement operator is applied once every min{2n, 100} generations (parameter π_R) to the best individual of the current population: the frequency decreases with n and the limit of 100 generations guarantees effects for instances with $n \geq 50$. For the remaining GAG parameters, we have experimentally set π_B to the 40% of the individuals, π_E to the 2% of the individuals, $\pi_M = 0.15$, $\pi_D = 100$, and $\pi_S = 5$.

Table 2 reports the results on VLSI benchmark and compares GAG with the CGA by Oliveira and Lorena (2002), where detailed results on both TOS and MOS are provided. Instance name and size are shown in the first column. Column λ is the threshold on MOS, corresponding to the minimum MOS value found by GAG. The same MOS value is also found by CGA and, for all the instances, it corresponds to proven optimal or best known (instance W4) MOS. The following columns summarize the results of 10 trials of CGA and GAG. SR_λ is the success rate, that is, the

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

percentage of trials obtaining a MOS = λ. TOS, Avg and Dev are, respectively, the best found TOS value, the average TOS value, and the percent standard deviation over the feasible sequences having MOS = λ. Note that Avg and Dev refer to the top five trials, in order to make it comparable to the results presented by Oliveira and Lorena (2002). *T*(s) is the average computational time, in seconds, over all the 10 trials (note that GAG and CGA use different hardware).

The B&C procedure has been run, with a time limit of 1 hour, with the aim of improving over the TOS value provided by GAG, or proving its optimality, under the constraint MOS ≤ λ: the last three columns of Table 2 report the obtained TOS values (proven optima in bold), the computational time needed to prove optimality or find the improved solution and the number of generated cuts. First, observe that, for two instances, namely Wli and V4000, CGA provides incorrect TOS values (in italics), as they are below the proven optimal solution value. For Wli, this is probably due to a typo in one of the result tables of Oliveira and Lorena (2002) (in a later table, the value 24 is reported). For V4000, recall that CGA is not designed to deal with constraints on MOS and so the value 53 for TOS is probably related to a gate sequence with MOS greater than 5 (in fact, the B&C procedure finds that the optimal TOS value is 51 if no threshold on MOS is considered—the related sequence has MOS value equal to 6). For all the remaining instances but one (V4090), GAG provides better TOS values. Moreover, GAG appears to be more reliable than CGA as it finds the best MOS value more frequently than CGA (for all instances but V4000) and it has lower average TOS value (except W4). Even if computational efficiency cannot be directly compared (CGA ran on a 266 MHz Intel Pentium processor), observe that GAG running times takes limited values (29.1 seconds in the worst case). We remark that the TOS values shown in Table 2 are obtained from feasible sequences, that is, the sequences whose MOS does not exceed λ. In fact, minimizing TOS is not equivalent to minimizing MOS, as shown by Linhares and Yanasse (2002), and GAG was able to find nonfeasible solutions with better TOS value: for example, trials on W4 obtained TOS = 1606 with MOS = 28, and TOS = 1514 with MOS = 30; one trial on V4000 obtained TOS = 51 with MOS = 6. The TOS value found by GAG, in four instances, has been proven to be optimal, under the restriction MOS ≤ λ, by the B&C algorithm.

The results on the remaining instances are shown in Tables 3 and 4. The former describes small-medium size instances (i.e., the related production matrices have less than 50 rows and columns), the latter to larger instances. GAG has been run 10 times for each instance. For these data sets, no results on TOS are available in the literature, and the values found by GAG have been compared with the optimal solutions or the lower bounds provided by the B&C procedure.

Each row of Table 3 reports average results for a class of instances from the data set indicated in the first column and having *n* and *m* within the ranges shown in the second and the third columns. The number of instances belonging to the class is in the fourth column. Again, λ is the threshold on MOS, corresponding to the minimum MOS value found by GAG and provided to the B&C procedure: this is always the optimum MOS (known from the literature, or obtained by applying the algorithm described in Chu and Stuckey, 2009), but in six instances, where it differs by one; furthermore, the success rate, that is the number of GAG trials finding the minimum MOS, is over the 90% on average, for all classes. The following columns describe GAG results: the average minimum TOS values and related percent standard deviation, the average computational time in seconds, the hit ratio (i.e., the percentage of instances for which GAG finds the optimal TOS value certified by the B&C procedure), and the average gap with respect to the lower bound determined by the B&C procedure. The last columns report the results of the B&C algorithm set with a computational time

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

Table 3
Results on classes of instances from literature and woodcutting industry

					GAG					B&C					
Instance	n	m	Number	λ	TOS	Dev (%)	T (s)	Hit (%)	Gap (%)	TOS	Opt (%)	Gap (%)	T (s)	Impr	Cuts
Wood. A[a]	13–21	20–31	4	5.75	35.25	0.64	0.07	100.0	0.00	35.25	100.0	0.00	26.32	0	848.2
Wood. B[a]	10–26	14–37	11	5.73	35.73	0.49	0.06	90.9	0.38	35.64	100.0	0.00	4.95	1 (1.0)	21,123.1
Class 1[b]	10–15	9–30	60	6.52	42.47	0.18	0.08	98.3	0.03	42.45	100.0	0.00	90.49	1 (1.0)	1,110.2
Class 2[b]	10–15	40–50	40	7.63	25.85	0.27	0.14	92.5	0.85	25.75	100.0	0.00	63.99	3 (1.3)	491.3
Class 3[b]	20–25	10–30	60	8.85	122.63	0.31	0.43	50.0	5.37	122.48	55.0	5.23	1894.57	5 (1.8)	198,765.3
Class 4[b]	20–25	40–50	40	9.25	107.25	0.39	0.66	37.5	9.01	107.23	37.5	8.98	2522.61	1 (1.0)	123,344.4
Class 5[b]	30–30	10–29	30	10.37	198.33	0.36	1.01	10.0	17.15	198.20	10.0	17.06	3265.14	2 (2.0)	423,841.9
Class 6[b]	30–30	40–50	20	11.70	204.90	0.45	1.47	0.0	29.17	204.90	0.0	29.17	3600.00	0	344,862.9
Shaw[c]	20–20	20–20	25	13.68	184.32	0.27	0.19	84.0	0.27	184.16	96.0	0.19	1202.48	4 (1.0)	202,160.6
Wilson[c]	20–50	10–50	12	19.00	737.92	0.04	1.09	66.7	9.38	737.83	75.0	9.14	1979.37	1 (1.0)	296,002.0

[a]SCOOP Team (2009). [b]Faggioli and Bentivoglio (1998). [c]Smith and Gent (2005).

Table 4
Results on individual instances from literature and woodcutting industry

				GAG						B&C			
Instance	n	m	λ	SR_λ (%)	TOS	Avg	Dev (%)	T(s)	Gap (%)	TOS	T(s)	Gap (%)	Cuts
A_FA_15	18	68	9	100.0	**52**	56.0	5.5	0.10	0.0	**52**	246.35	0.0	3340
A_FA_2	19	75	11	100.0	**58**	58.0	0.0	0.13	0.0	**58**	676.41	0.0	2404
A_FA_12	20	75	9	100.0	**48**	48.0	0.0	0.15	0.0	**48**	26.57	0.0	1962
A_FA_6	21	79	13	100.0	**91**	91.0	0.0	0.19	0.0	**91**	591.80	0.0	3191
A_FA_11	28	99	11	100.0	**87**	87.0	0.0	0.40	0.0	**87**	2361.89	0.0	126,903
A_FA_8	28	82	12	100.0	112	112.7	0.4	0.65	1.8	**110**	444.63	0.0	7493
A_FA_1	37	107	12	100.0	184	187.0	1.6	4.08	30.3	184	3600.00	30.3	264,938
A_FA_13	37	134	18	100.0	304	306.1	1.2	2.95	57.2	304	3600.00	57.2	225,614
B_REVAL_145	49	60	7	100.0	129	129.7	0.5	1.42	0.8	**128**	119.97	0.0	28,449
NWRS7	60	25	10	100.0	**316**	316.2	0.1	4.46	0.0	**316**	891.11	0.0	293,162
NWRS8	60	25	16	100.0	591	591.0	0.0	7.53	26.7	591	3600.00	26.7	2,440,244
GP5	100	100	95	100.0	9341	9342.9	0.0	15.09	–	9341	–	–	–
GP6	100	100	75	100.0	7359	7359.0	0.0	27.23	–	7359	–	–	–
GP7	100	100	75	100.0	7366	7366.0	0.0	23.11	–	7366	–	–	–
GP8	100	100	60	100.0	5858	5858.0	0.0	37.93	–	5858	–	–	–
SP3	75	75	34	30.0	1622	1644.7	1.2	26.28	177.4%	1622	3600.00	177.4	376,456
SP4	100	100	53	10.0%	3450	3450.0	0.0	56.79	–	3450	–	–	-

–, no lower bound after 1-hour computation (TOS refers to the value initially provided by GAG).

limit of 1 hour: the average best TOS value, the percentage of instances with certified optimum TOS (subject to the λ threshold), the average gap, the average computational time in seconds, the number of instances where the B&C improves the TOS value provided by GAG, with the average TOS value reduction in parenthesis, and, finally, the number of generated cuts.

GAG is very fast in handling all the instances of Table 3 (the computational time is less than 1.47 seconds) and also very reliable, as the TOS standard deviation is very small (recall that the

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

MOS is always the optimal one, but in a few cases). The B&C procedure is able to solve all the small-medium size instances from the woodcutting industry to optimality, as well as the literature instances up to $n = 15$, in a reasonable amount of time. For larger values of n, the number of optimal solutions found decreases and becomes zero for one class of instances with $n = 30$ and $m \geq 40$, with no improvements of the GAG solutions within the time limit of 1 hour (note that increasing values of n have a larger impact on the performance than m). The hit ratio of GAG follows the same trend. In fact, in most of the optimally solved instances, the B&C algorithm certifies the optimality of the GAG solutions and, in 18 cases, it improves the solutions provided by GAG by one or two units, sometimes determining the optimal TOS value. Looking at the gap values, we note that GAG provides optimal or near-optimal solutions for smaller instances, while the gap is larger for instances with n over 25. This is probably due to the available dual bound, rather than to the quality of the solutions provided by GAG: in fact, the B&C gap and the computational time show a degradation of the performance, due to the nature of the exact procedure.

Table 4 reports the results on the largest instances from the woodcutting benchmark and on other large instances from literature. After the columns describing the instance name and size, column λ reports the λ threshold on MOS found by GAG, and column SR_λ the number of times GAG finds a MOS $= \lambda$. Note that λ corresponds to the proven optimal MOS, certified by applying the algorithm described in Chu and Stuckey (2009). The following columns report, with respect to GAG, the minimum TOS value, average TOS value, and related percent standard deviation, the computational time and the gap with respect to the lower bound determined by the B&C procedure. The last columns are the TOS values, computational time, gap, and the number of generated cuts related to the B&C run with a time limit of 1 hour. The certified optimal value of TOS (under the constraint MOS $\leq \lambda$) are shown in bold.

For the threshold λ, GAG always provides the optimal MOS, which is found by all the 10 trials for all instance but SP3 and SP4. For some instances, the values of TOS correspond to optimal solutions certified by the B&C algorithm, or to near-optimal solutions (the gap with respect to the lower bound is less than 1.8%). In the remaining cases, the large gaps seems to be related to the loose lower bound provided by the B&C procedure within the time limit established, as discussed previously. Considering the instances where the available dual bounds certify the quality of the solution, the standard deviation is over 1.6% just for one instance, showing the reliability of GAG in these cases. The B&C procedure is able to solve to optimality almost all the real instances from the woodcutting companies that we have taken into account, even for $n = 49$: just two instances with large m remain open with a remarkable gap. As for the literature instances, one of them is solved to optimality, while, in the other cases, the B&C algorithm presents large gaps or, even, it is not able to solve the linear relaxation within 1 hour, thus confirming, together with the results shown in Table 3, that the proposed exact procedure in its current implementation is probably better suited for small-medium size instances.

5. Conclusions

We presented a genetic approach (GAG) and a B&C procedure for GMCCP, a pattern-sequencing problem dealing with TOS minimization under restrictions on MOS. GAG defines the fitness function as a dynamic weighted sum of TOS, MOS, and other new performance indicators, to

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

take into account both global and local features of the pattern sequences; it also implements several speeding-up tricks, including incremental neighborhood and offspring evaluation. The implemented B&C procedure is, to our best knowledge, the first algorithm designed to find proven optimal TOS: it is based on the properties of C1P matrices and it is flexible enough to accommodate different objectives or performance constraints. Results on real and literature instances show that GAG usually outperforms the other algorithms presented in the literature. B&C is able to provide optimal solutions or small gaps for small instances and some real medium size instances. In these cases, B&C proves the optimality (or near optimality) of the solution found by GAG. Ongoing research proceeds in different directions: further improvement of the incremental neighborhood and offspring evaluation; automatic parameter tuning using available tools (e.g., Birattari, 2009); dynamic control of parameters, in particular weight settings, switching from the current static phase hops, to triggering intensification and diversification through landscape and convergence analysis; improvement of B&C efficiency on large instances and implementation of exact separation procedures.

References

Achterberg, T., 2009. SCIP: solving constraint integer programs. *Mathematical Programming Computation* 1, 1, 1–41.

Baptiste, P., 2005. Simple MIP formulations to minimize the maximum number of open stacks. In Smith, B.M. and Gent, I.P. (eds), *Proceedings of IJCAI'05—Constraint Modelling Challenge 2005*, Edinburgh, pp. 9–13.

Becceneri, J.C., Yanasse, H.H., Soma, N.Y., 2004. A method for solving the minimization of the maximum number of open stacks problem within a cutting process. *Computers and Operations Research* 31, 2315–2332.

Birattari, M., 2009. *Tuning Metaheuristics: A Machine Learning Perspective*, Vol. 197. Springer Verlag, Berlin.

Booth, K.S., Lueker, G.S., 1976. Testing for the consecutive ones property, interval graphs, and graph planarity using PQ-tree algorithms. *Journal of Computer System Science* 13, 335–379.

Chu, G., Stuckey, P.J., 2009. Minimizing the maximum number of open stacks by customer search. *Lecture Notes in Computer Science* 5732, 242–257.

De Giovanni, L., Massi, G., Pezzella, F., 2013. An adaptive genetic algorithm for large-size open stack problems. *International Journal of Production Research* 51, 3, 682–697.

Dyson, R., Gregory, A., 1974. The cutting stock problem in the flat glass industry. *Operational Research Quarterly* 25, 41–54.

Faggioli, E., Bentivoglio, C., 1998. Heuristic and exact methods for the cutting sequencing problem. *European Journal of Operational Research* 110, 3, 564–575.

Förster, H., Wäscher, G., 1998. Simulated annealing for order spread minimization problem in sequencing cutting patterns. *European Journal of Operational Research* 110, 271–281.

Hu, Y.H., Chen, S.J., 1990. GM_Plan: a gate matrix layout algorithm based on artificial intelligence planning techniques. *IEEE Transactions on Computer-Aided Design* 9, 836–845.

Lin, S., Kernighan, B.W., 1973. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research* 21, 2, 498–516.

Linhares, A., Yanasse, H.H., 2002. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers & Operations Research* 29, 1759–1772.

Lopes, I.C., Valério de Carvalho, J.M., 2010. Using interval graphs in an order processing optimization problem. *World Congress on Engineering WCE 2010*, Vol. III. London, 30 June–2 July, 1722–1728.

Madsen, O., 1988. An application of travelling-salesman routines to solve pattern-allocation problems in the glass industry. *Journal of Operational Research Society* 39, 249–256.

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies

Möhring, R., 1990. Graph problems related to gate matrix layout and PLA folding. *Computing* 7, 17–51.

Oliveira, A.C.M., Lorena, L.A.N., 2002. A constructive genetic algorithm for gate matrix layout problems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 21, 8, 969–974.

Oliveira, A.C.M., Lorena, L.A.N., 2006. Pattern sequencing problems by clustering search. *Lecture Notes in Computer Science* 4140, 218–227.

Oliver, I., Smith, D., Holland, J., 1987. A study of permutation crossover operators on the traveling salesman problem. *Proceedings of the Second International Conference on Genetic Algorithms and Their Application*, Cambridge, MA, pp. 224–230.

Oswald, M., Reinelt, G., 2003. Constructing new facets of the consecutive ones polytope. In Jünger, M., Reinelt, G., and Rinaldi, G. (eds) *Combinatorial Optimization—Eureka, You Shrink! Papers Dedicated to Jack Edmonds, 5th International Workshop, Aussois, 2001*, Vol. 2570 of *LNCS*, Springer-Verlag, Berlin, pp. 147–157.

Oswald, M., Reinelt, G., 2004. Computing optimal consecutive ones matrices. In Grötschel, M. (ed.) *The Sharpest Cut, The Impact of Manfred Padberg and His Work*, *Optimization*, MPS/SIAM, Philadelphia, PA, pp. 173–184.

Respício, A., Captivo, M.E., 2005. Bi-objective sequencing of cutting patterns—an application for the paper industry. In Ibaraki, T., Nonobe, K., and Yagiura, M. (eds) *Metaheuristics: Progress as Real Problem Solvers*, Springer, New York, pp. 227–241.

SCOOP Team, 2009. Sheet cutting and process optimization for furniture industry—SCOOP Project. Available at http://www.scoop-project.net (accessed May 27, 2013).

Smith, B. M., Gent, I. P. (eds), 2005. *Proceedings of IJCAI'05—Constraint Modelling Challenge 2005*, Edinburgh.

Tucker, A., 1972. A structure theorem for the consecutive 1's property. *Journal of Combinatorial Theory Series B* 12, 153–162.

Yuen, B.J., 1995. Improved heuristics for sequencing cutting patterns. *European Journal of Operational Research* 87, 1, 57–64.

© 2013 The Authors.
International Transactions in Operational Research © 2013 International Federation of Operational Research Societies